Examen parcial - Orientación a Objetos I - 13/12/2025

Comentarios generales:

- En cada hoja, escriba apellido, nombre, legajo o dni, y número de hoja / el total de hojas entregadas.
- Tenga en cuenta que los 4 ejercicios son **requeridos para aprobar**. No deje ninguno en blanco.
- Debe entregar al menos una hoja con sus datos, aunque no haya completado todo lo pedido.

Ejercicio 1

En un centro de esquí se organizan diversas actividades donde las personas pueden participar para practicar. Cada actividad está planificada para una fecha y horario determinado y cuenta con un nombre que la identifica y una breve descripción de su propuesta.

Las actividades de práctica se realizan en una pista de esquí. Estas actividades tienen un máximo de participantes. Cada pista utilizada tiene un nombre y un nivel de dificultad (un valor entero entre 1 y 10), y puede ser utilizada en varias actividades a lo largo de la temporada. La duración de las mismas depende del número de participantes inscritos y se calcula como 10 minutos por cada participante.

Otras actividades adoptan un formato de taller comunitario. Las personas se reúnen para compartir experiencias, intercambiar conocimientos y practicar de manera colaborativa. Cada taller cuenta con un organizador, que es una persona que puede participar en otras actividades y se encarga de coordinar el taller. Cada taller tiene un lugar de encuentro y una duración en minutos, que se decide al momento de su planificación. Los talleres no tienen límite de participantes.

Las pistas mantienen un registro de incidentes ocurridos durante las actividades que se realizan en ellas. Cada incidente incluye una breve descripción de lo sucedido, la fecha y hora en que ocurrió, y la persona involucrada. A lo largo del tiempo, cada pista puede acumular múltiples incidentes asociados.

El sistema debe permitir las siguientes funcionalidades:

1. **Agregar un participante a una actividad:** dadas una actividad y un participante, se inscribe al participante en la actividad. La inscripción se realiza siempre que no se haya alcanzado el máximo de participantes (cuando corresponda).
2. **Retornar la duración de una actividad:** Para las prácticas, la duración se calcula como 10 minutos por participante inscripto. Para los talleres comunitarios, la duración en minutos definida al planificar el taller.
3. **Listar participantes de una actividad:** Retorna todos los participantes inscriptos en una actividad, ordenados alfabéticamente por nombre.
4. **Listar talleres organizados por una persona:** Dada una persona, retorna todos los talleres que haya organizado ordenados por fecha.
5. **Registrar un incidente en una pista:** Permite registrar que una persona tuvo un incidente ocurrido en una pista participando de una actividad.
6. **Listar incidentes de una pista:** Retorna todos los incidentes registrados en una pista, ordenados cronológicamente.

Tarea:

1. Realice un diagrama de clases UML donde identifique las clases del dominio, atributos, y las relaciones con sus roles y cardinalidades según corresponda. No debe incluir comportamiento.

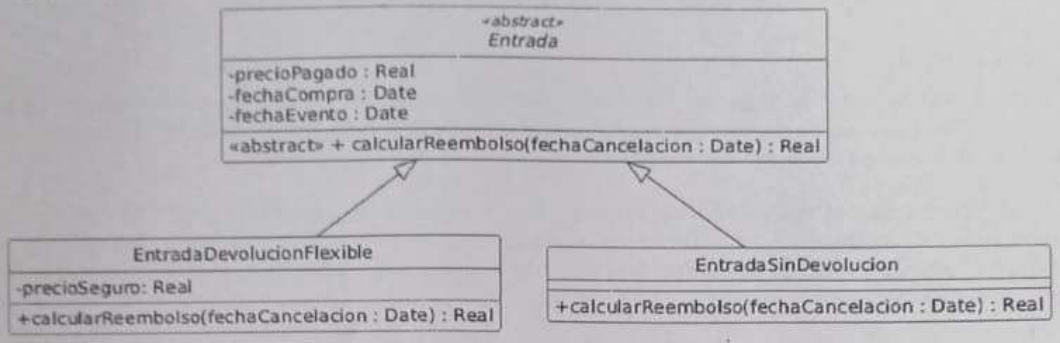
NOTA: Asegúrese que su modelo permite resolver todas las funcionalidades pedidas.



Ejercicio 2

Orientación a Objetos I - 13/12/2025

El diagrama de clases de UML que se muestra a continuación documenta parte del diseño de entradas a eventos. Actualmente existen dos tipos de entradas: EntradaConDevolucionFlexible y la EntradaSinDevolucion. Cada entrada registra el precio pagado, la fecha de compra y la fecha del evento; además las entradas con devolución flexible cuentan con un seguro, que es un monto adicional que se paga para permitir un reembolso.



Los compradores tienen la posibilidad de calcular el monto de reembolso en caso de querer devolver su entrada. La entrada flexible permite recuperar la totalidad de lo pagado (el precio pagado y el seguro) si la cancelación ocurre dentro de los 3 días de comprada. Si la cancelación ocurre después de ese período, pero antes del evento, el reembolso es del 50% del precio pagado. En cambio, la entrada sin devolución retorna 0, independientemente de cuándo se solicite la cancelación.

```
public class EntradaDevolucionFlexible extends Entrada {
  public EntradaDevolucionFlexible(double precioPagado,
     LocalDate fechaCompra, LocalDate fechaEvento,
     double precioSeguro) { ... }

  public double calcularReembolso(LocalDate fechaCancelacion){
    long diasDesdeCompra =  ChronoUnit.DAYS
                        .between(this.fechaCompra, fechaCancelacion);
    long diasRestantes = ChronoUnit.DAYS
                        .between(fechaCancelacion,this.fechaEvento);
    if (diasRestantes <= 0 ) { return 0.0; }  // ya pasó el evento
    if (diasDesdeCompra < 3) {
      return this.precioPagado + this.precioSeguro;
    }
     return this.precioPagado * 0.5; // mitad del precio
    }
}
```

```
public class EntradaSinDevolucion
                        extends Entrada {
   public EntradaSinDevolucion
      (double precioPagado,
      LocalDate fechaCompra,
      LocalDate fechaEvento)
      { ... }

   public double calcularReembolso (
        LocalDate fechaCancelacion) {
     return 0.0;
   }
}
```

Tareas:

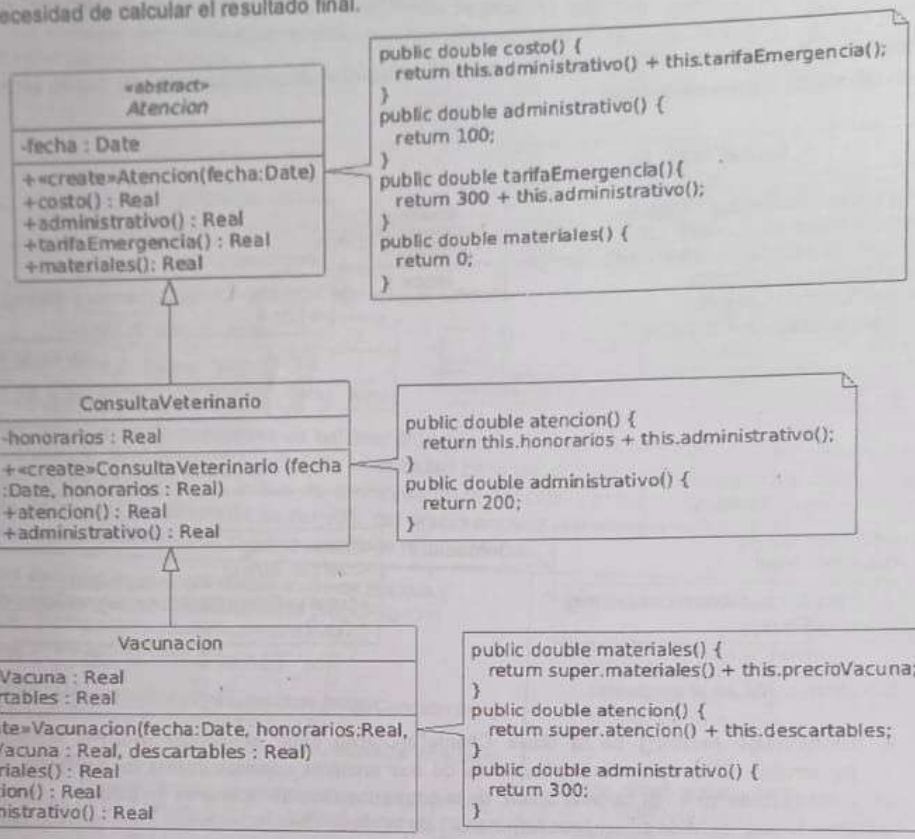
1. Rediseñe lo necesario para desacoplar la entrada de su política de reembolso, de forma que se pueda cambiar esa política de forma dinámica. Delegue polimórficamente en la política asignada el cálculo del monto a reembolsar. Realice un diagrama de clases UML completo de su diseño (incluyendo los métodos y constructores).
2. Reimplemente en Java respetando el nuevo diseño. Implemente todos los métodos necesarios para garantizar el correcto funcionamiento y permitir la creación coherente de los objetos.
3. Instancie una entrada con los datos que considere apropiados y una política de cancelación flexible. Calcule su reembolso y guárdelo en una variable. Luego, cambie la política de la entrada creada anteriormente a sin devolución y calcule nuevamente el reembolso
4. Diseñe los casos de test del método calcularReembolso para la política de reembolso flexible, indicando las particiones y valores de borde.

Ayuda: Para instanciar un objeto LocalDate, puede usar `LocalDate.of(year, month, day)`

Ejercicio 3

Orientación a Objetos I - 13/12/2025

Dado el siguiente modelo, y los fragmentos de código mostrados, complete el cuadro con los valores correspondientes al resultado de evaluar la sentencia indicada. Puede dejar la expresión (por ej. 100 + 200) sin necesidad de calcular el resultado final.



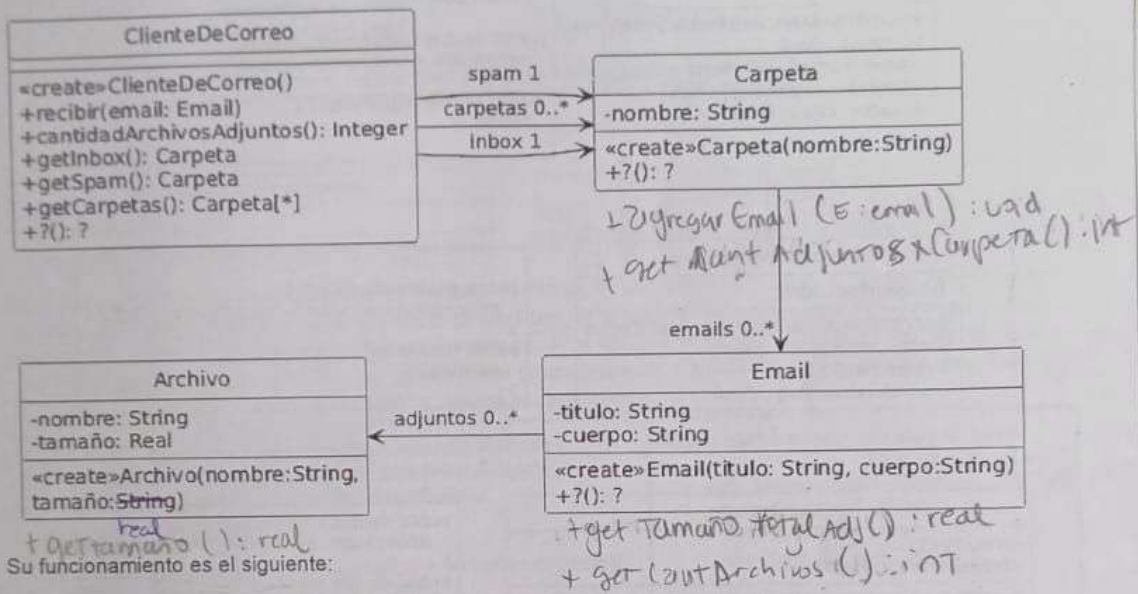
LocalDate fecha = LocalDate.of(2025, 9, 21)
Vacunacion vacuna = **new** Vacunacion(fecha, 1500, 800, 500);
ConsultaVeterinario consulta = **new** ConsultaVeterinario(fecha, 200);

Sentencia a evaluar	Resultado
`vacuna.tarifaEmergencia();`	
`vacuna.materiales();`	
`vacuna.atencion();`	
`consulta.costo();`	

Ejercicio 4

Orientación a Objetos I - 13/12/2025

El diagrama de clases de UML que se muestra a continuación documenta parte del diseño simplificado de un cliente de correo electrónico.



Su funcionamiento es el siguiente:

- El mensaje **recibir()** de la clase ClienteDeCorreo debe almacenar el email recibido como parámetro. Si la suma de los tamaños de sus archivos adjuntos supera los 8000 KB, el correo debe guardarse en la carpeta *spam*; de lo contrario, debe almacenarse en *inbox*.
- El mensaje **cantidadArchivosAdjuntos()** de la clase ClienteDeCorreo debe retornar la cantidad total de archivos adjuntos presentes en el cliente de correo, considerando los correos almacenados en cualquiera de sus carpetas.

Tareas:

1. Respetando las firmas definidas, implemente los constructores de las clases ClienteDeCorreo y Carpeta.
2. Implemente los métodos recibir() y cantidadArchivosAdjuntos() junto con los métodos adicionales que sean necesarios para soportar ese funcionamiento. Utilice Streams cuando resulte apropiado.
3. ¿Es posible cambiar el nombre de la carpeta inbox a "entrada" y que el ClienteDeCorreo se siga comportando de la misma manera? En caso de ser posible, utilice el mensaje setNombre de la clase Carpeta (asuma ya está implementado) para realizar este cambio; si no es posible, explique brevemente los motivos.